



Introduction to Embodied AI

Lecture 4: Camera Models & Camera Calibration

Junwei Liang
AI Thrust
HKUST (Guangzhou)

Logistics - Project 1 Paper list is out!!

- Choose 1 paper to present
- You will need to start forming groups (1-3 ppl) now. **Post your teammates on the Discussion section on Canvas**

Course website:

<https://hkust-aiaa4220.github.io/fall2025-L01/ecs.html>

Canvas:

<https://hkust-gz.instructure.com/courses/2103>

Paper list:

<https://hkust-aiaa4220.github.io/fall2025-L01/project1.html>

Numbering	theme	paper title	conference	Note
R1.1	vision transformers	An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale	ICLR 2021	
R1.2	vision transformers	Hierarchical Vision Transformer using Shifted Windows	ICCV 2021	
R1.3	vision transformers	Multiscale Vision Transformers	ICCV 2021	
R2.1	Visual Localization	Benchmarking Visual Localization for Autonomous Navigation	WACV 2023	
R2.2	Visual Localization	LASER: LATent Space Rendering for 2D Visual Localization	CVPR 2022	
R2.3	Visual Localization	From Coarse to Fine: Robust Hierarchical Localization at Large Scale	CVPR 2019	
R3.1	SLAM	BEV-SLAM: Building a Globally-Consistent World Map Using Monocular Vision	IROS 2022	
R3.2	Planning and control	NEAT: Neural Attention Fields for End-to-End Autonomous Driving	ICCV 2021	
R4.1	Navigation	From Cognition to Precognition: A Future-Aware Framework for Social Navigation	ICRA 2025	Project 3!
R4.2	Navigation	Room-Across-Room: Multilingual Vision-and-Language Navigation with Dense Spatiotemporal Grounding	EMNLP 2020	
R4.3	Locomotion	Omni-Perception: Omnidirectional Collision Avoidance for Legged Locomotion in Dynamic Environments	CoRL 2025	Oral
R4.4	Navigation	Prioritized Semantic Learning for Zero-shot Instance Navigation	ECCV 2024	
R5.1	Manipulation	Do As I Can, Not As I Say: Grounding Language in Robotic Affordances	2022	
R5.2	Manipulation	GLOVER++: Unleashing the Potential of Affordance Learning from Human Behaviors for Robotic Manipulation	CoRL 2025	
R5.3	Manipulation	Mitigating the Human-Robot Domain Discrepancy in Visual Pre-training for Robotic Manipulation	CVPR 2025	

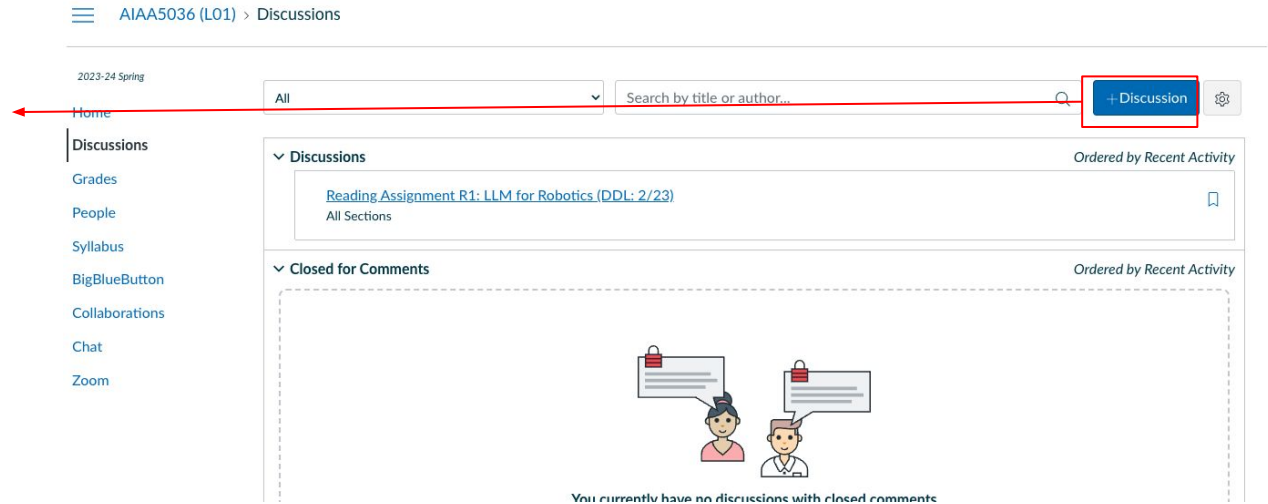
Logistics - Project 1 Paper list is out!!

- Project 1: Paper Presentation (20%)
 - You will select 1 paper from our list of Embodied AI research to present. This should show that you understand the paper, and you could present it to students who do not know the paper well. **You have 10 minutes to present and 5 minutes to handle at least one question (from TA/peers/teacher)**
 - Will be released on 9/8
 - Full schedule
 - P1 group due on 9/15
 - P1 presentation order release on 9/22
 - P1 presentation on 10/13
 - Can be done in groups of 1-3 people. **Score multiplier: 1.1x for solo, 1.05x for two people and 1.0x for three people group.**
 - Project 1 is worth 20% credit.
 - You will be graded based on Understanding & Content (40%), Presentation & Communication (30%), Critical Analysis & Discussion (30%), by the teacher, TAs, and peers.
 - **You should learn this paper well enough to handle questions!**

Logistics - Project 1 Paper list is out!!

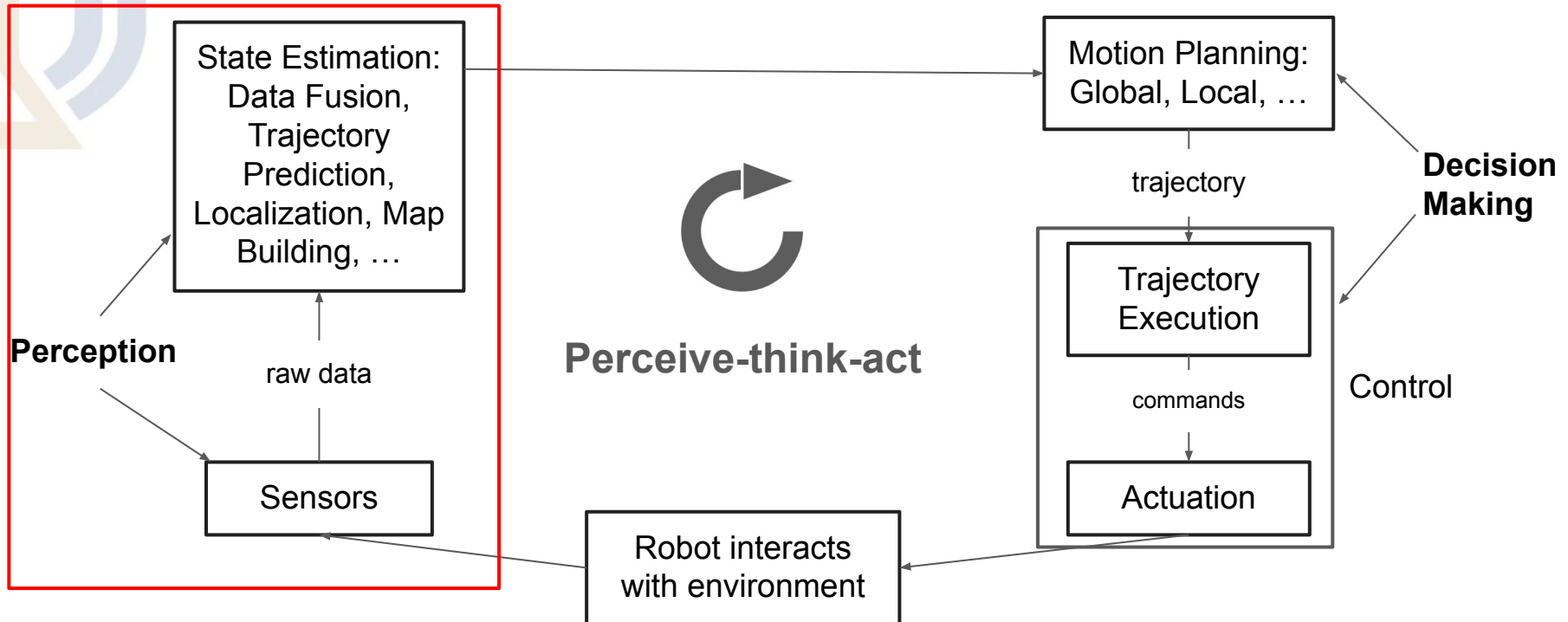
- How-To

- Discussions section, click “+Discussion”.
- **Write your teammates** (and the paper you choose)



Recap - The Autonomous AI Paradigm

- The Perceive-think-act circle

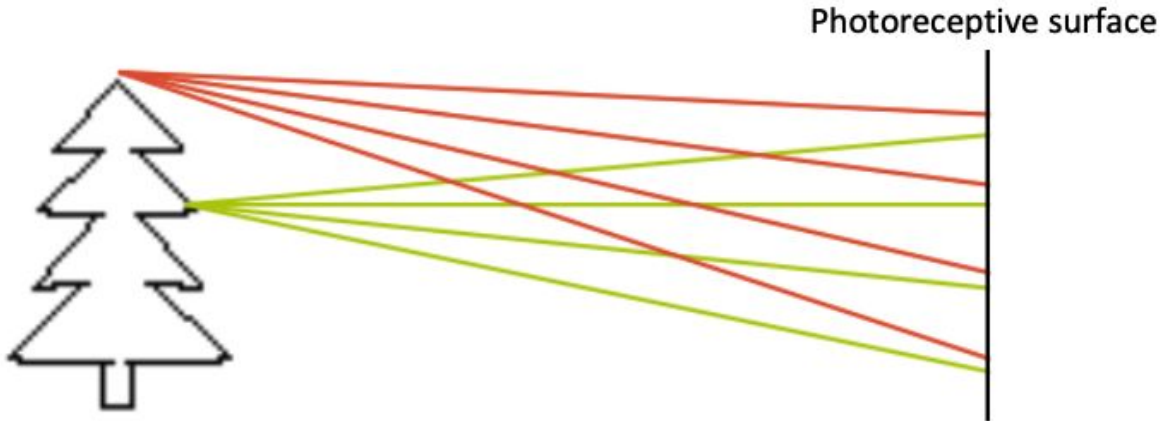


Agenda - Camera Models & Camera Calibration

- Objectives
 - Learn about cameras, camera models and camera calibration
 - So you can put what you see/what the camera sees into the 3D world!
- The tasks
 - Camera models
 - Perspective projection
 - Camera calibration
 - A note on 3D reconstruction

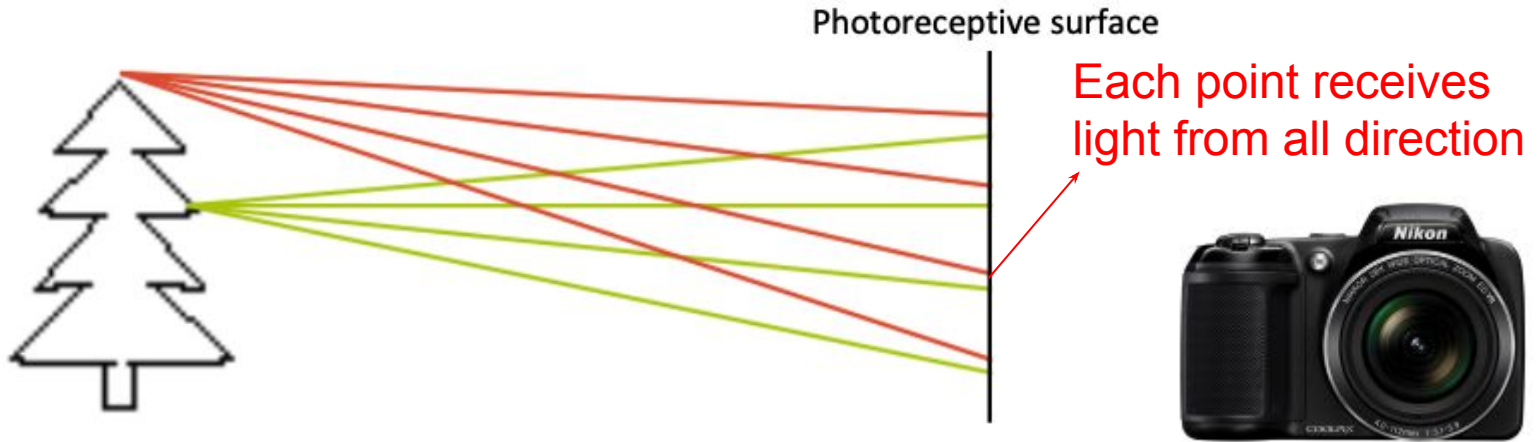
Cameras

- How to capture an image of the world
 - (Digital) Cameras -> capture light -> converted to digital image
 - Light is reflected by the object and scattered in all directions
 - if we simply add a photoreceptive surface, the captured image will be extremely blurred
 - why?



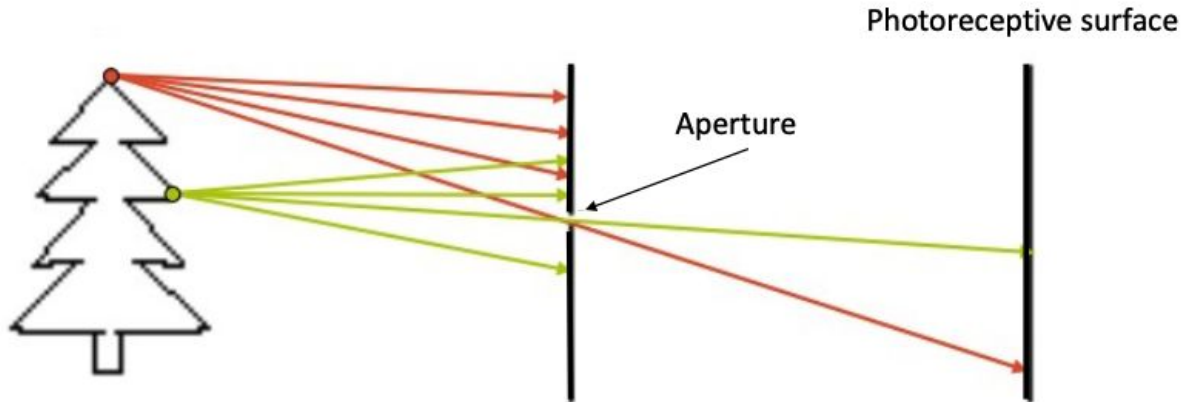
Cameras

- How to capture an image of the world
 - (Digital) Cameras -> capture light -> converted to digital image
 - Light is reflected by the object and scattered in all directions
 - if we simply add a photoreceptive surface, the captured image will be extremely blurred
 - why?



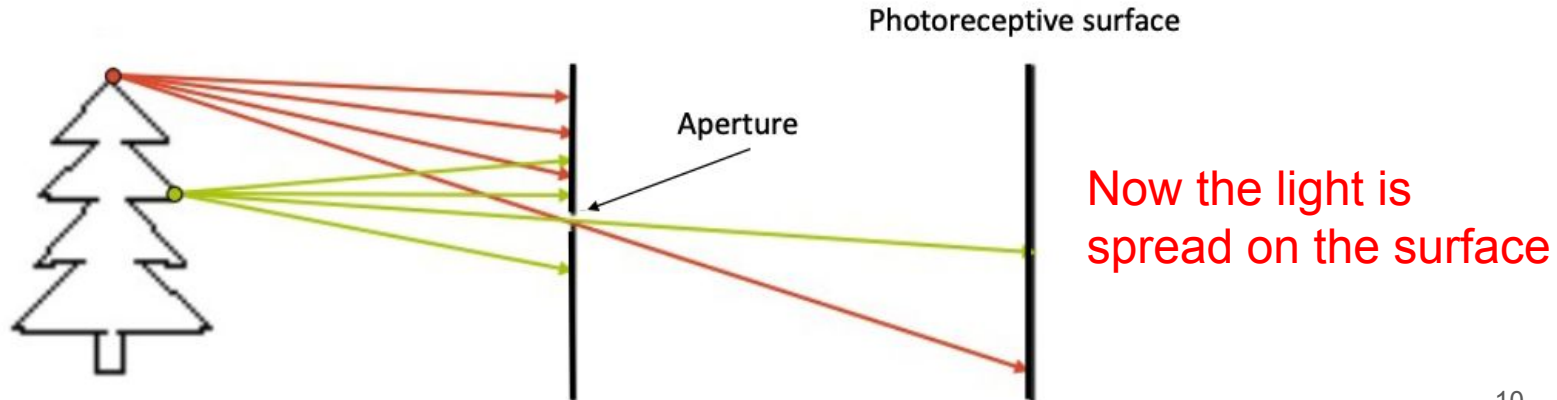
Pinhole Camera

- How to capture an image of the world
 - Add a barrier to block off most of the light/rays
 - The light can only come through the aperture in the center
 - Pinhole camera: a camera without a lens but with a tiny aperture, a pinhole



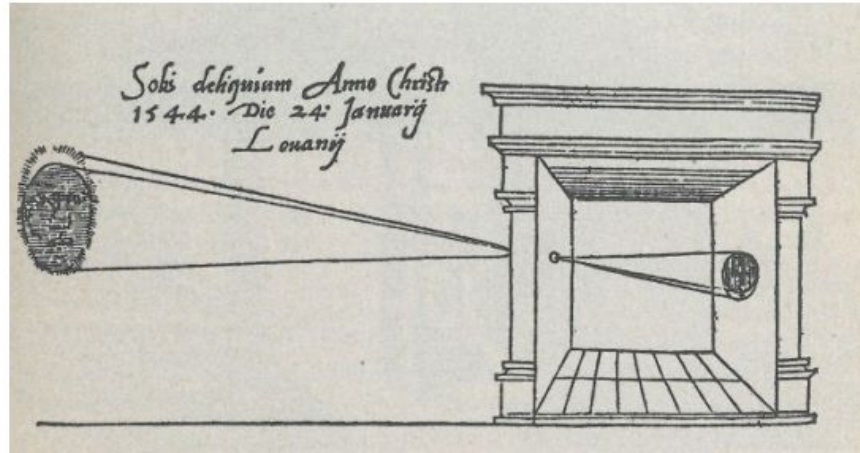
Pinhole Camera

- How to capture an image of the world
 - Add a barrier to block off most of the light/rays
 - The light can only come through the aperture in the center
 - Pinhole camera: a camera without a lens but with a tiny aperture, a pinhole

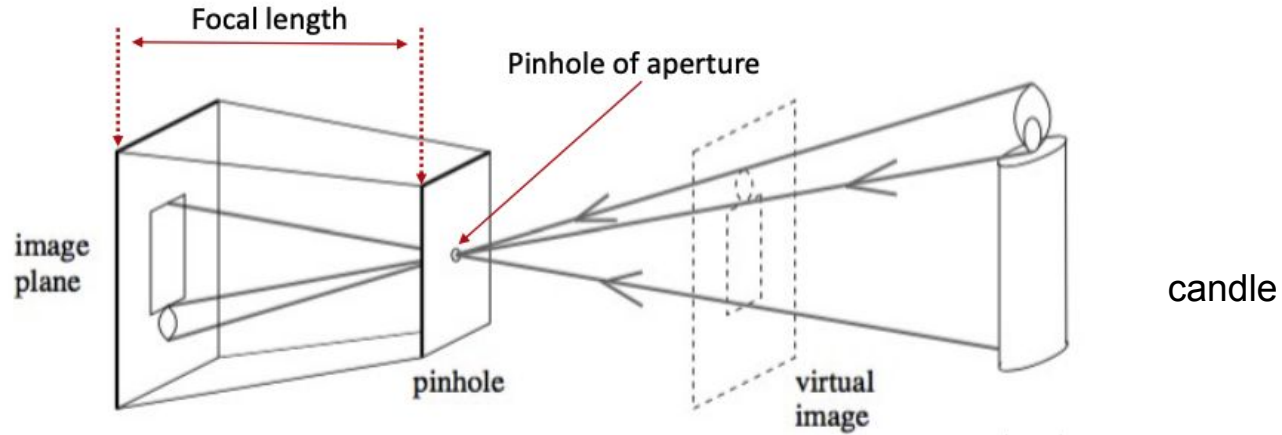


Pinhole Camera

- A long history
 - First clear description from Leonardo Da Vinci (1502)
 - Oldest known published drawing of a camera obscura by Gemma Frisius (1544)

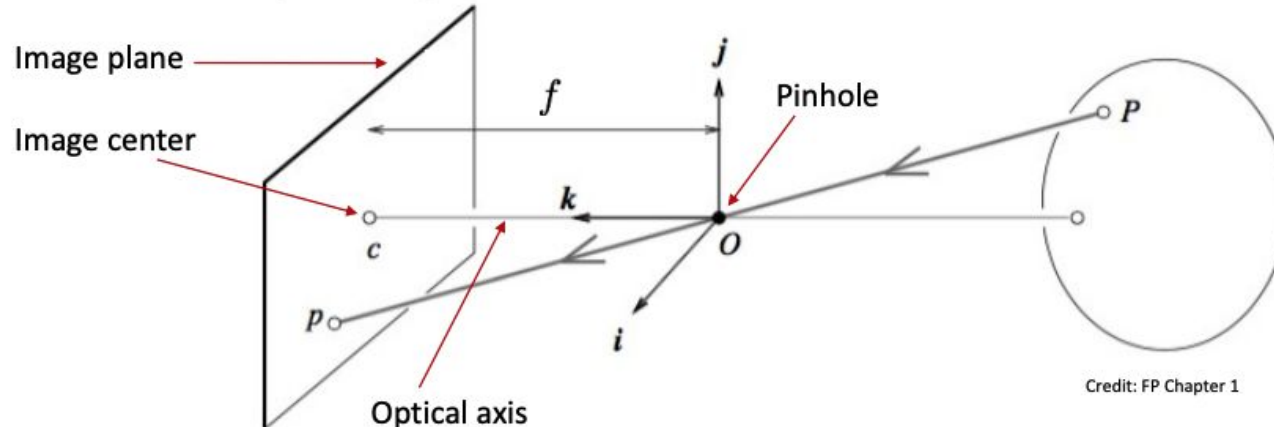


Pinhole Camera



- Perspective projection creates *inverted images*
- Sometimes it is convenient to consider a virtual image associated with a plane lying in front of the pinhole
- Virtual image is equivalent to the actual one except for on scale

Pinhole Camera



- How to project a point in the world to the image plane

Suppose we know P in the world frame

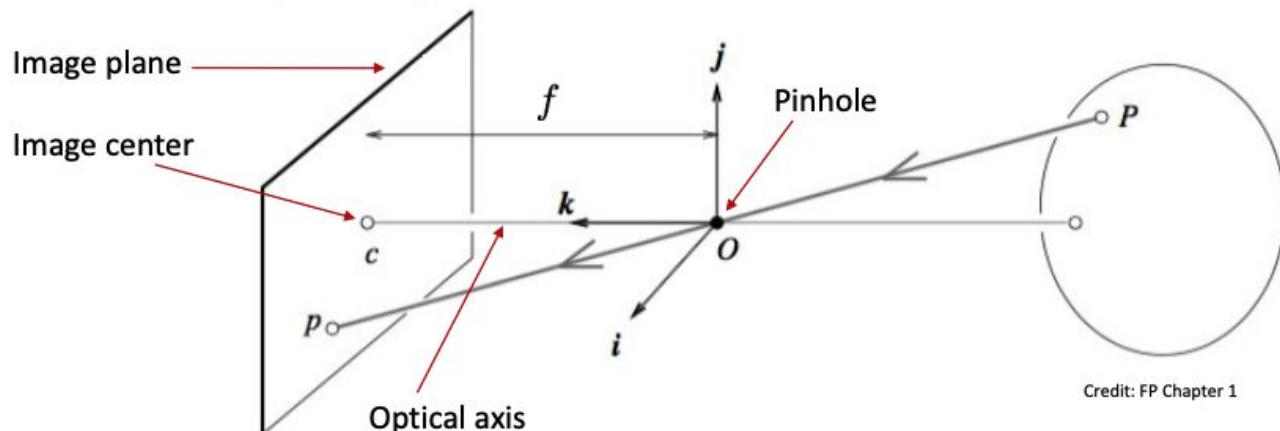
$$P = (X, Y, Z)$$

Perspective

$$p = (x, y, z)$$

How to get p in the image frame (3D, O is the origin)

Pinhole Camera



$$P = (X, Y, Z)$$

Perspective

$$p = (x, y, z)$$

- How to project a point in the world to the image plane

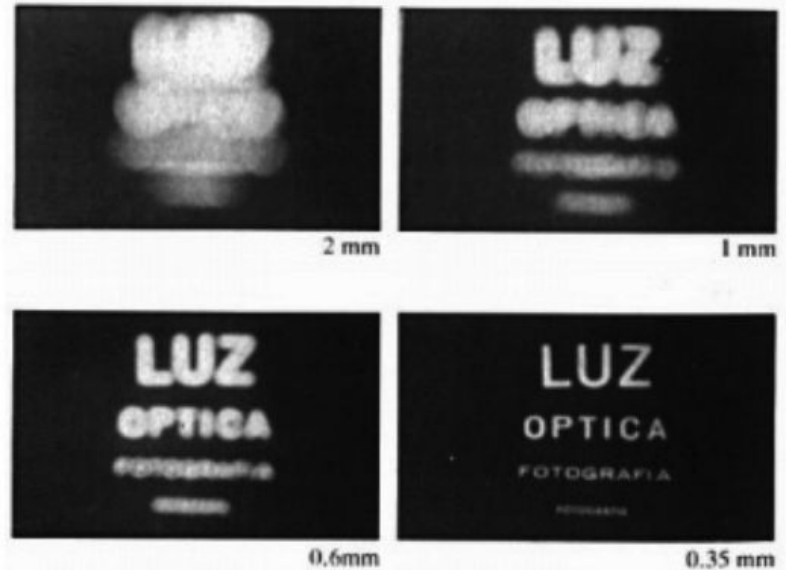
- Since P , O , and p are collinear (on the same line): $\overline{Op} = \lambda \overline{OP}$

$$\begin{cases} x = \lambda X \\ y = \lambda Y \\ z = \lambda Z \end{cases} \Leftrightarrow \lambda = \frac{x}{X} = \frac{y}{Y} = \frac{z}{Z} \Rightarrow \begin{cases} x = f \frac{X}{Z} \\ y = f \frac{Y}{Z} \end{cases}$$

Will use
this later

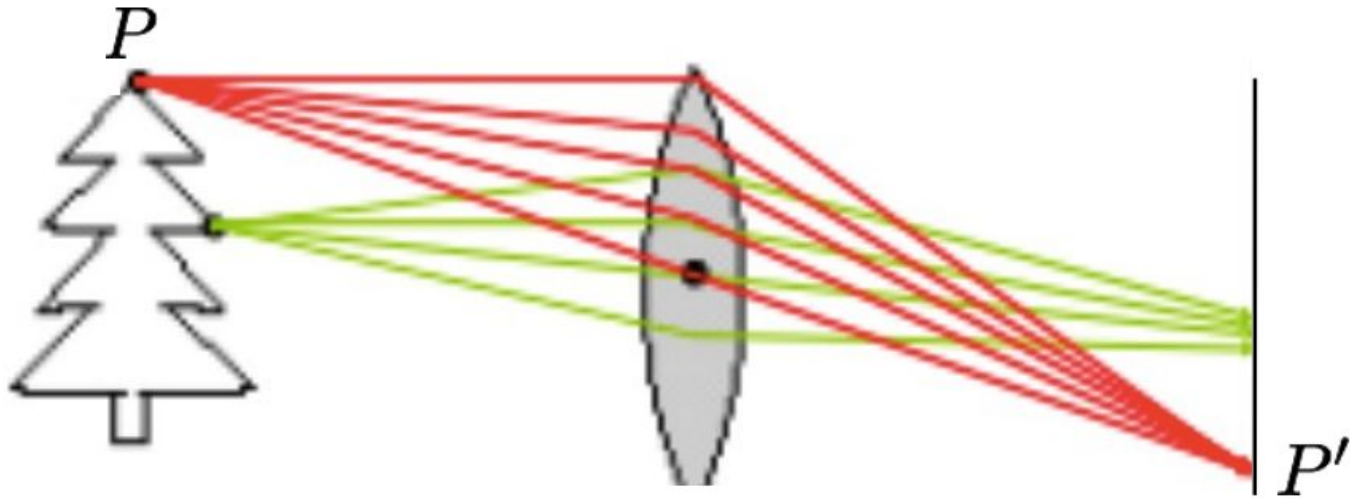
Issues with Pinhole Camera

- Larger hole: more lights passing through the pinhole -> blurry
- Smaller: darker
- Solution: add a lens to replace the pinhole

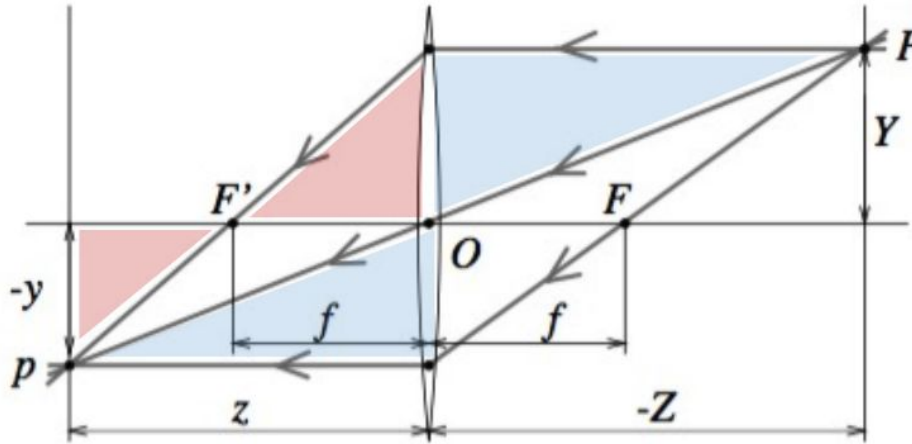


Lenses

- Lens: an optical element (glass) that focuses light by means of refraction

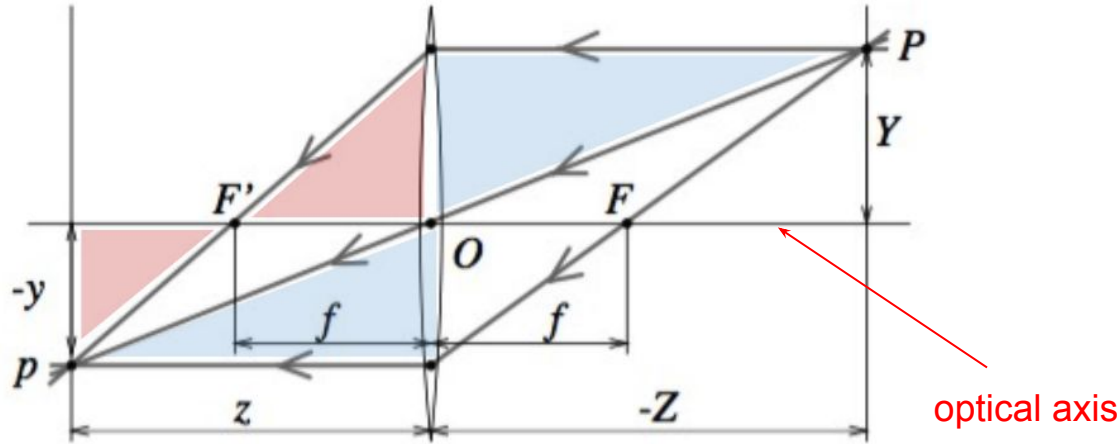


Thin Lens Model



- Key properties
 - Rays passing through O (lens center) are not refracted
 - Rays parallel to the optical axis are focused on the focal point F'
 - All rays passing through P are focused by the lens on the point p

Thin Lens Model



So p receives more rays from P compared to pinhole

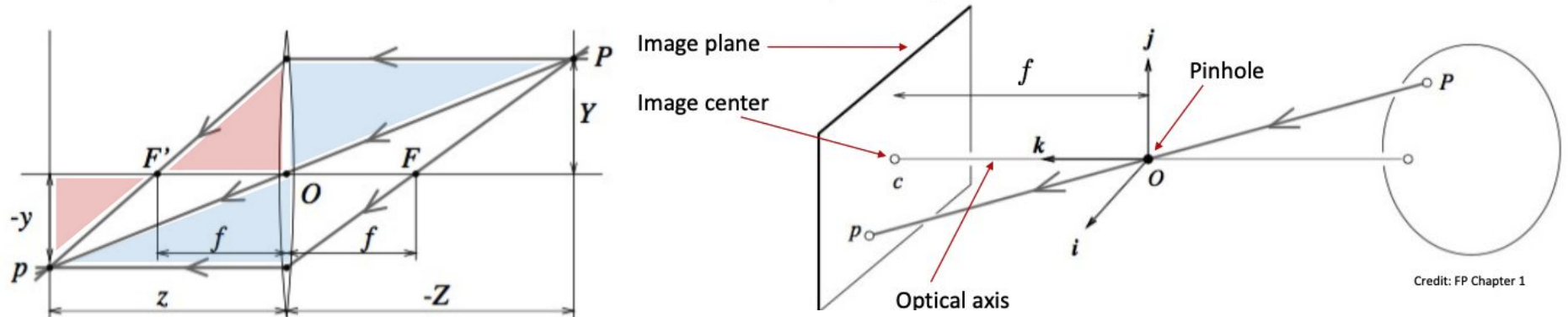
- Key properties

- Rays passing through O (lens center) are not refracted
- Rays parallel to the optical axis are focused on the focal point F'
- All rays passing through P are focused by the lens on the point p

Thin Lens Model

- Notes

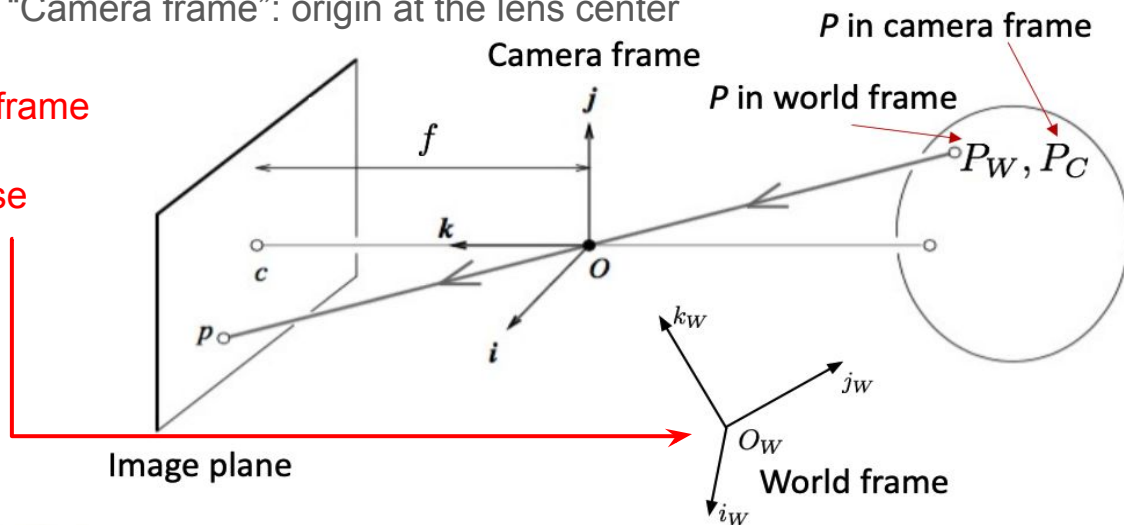
- If we consider z as the focal length, the perspective transformation equation is the same between thin lens model and pinhole model
- We ignore the depth of field effect for the thin lens model for simplicity
 - Assuming camera is focused at infinite far



Perspective Projection

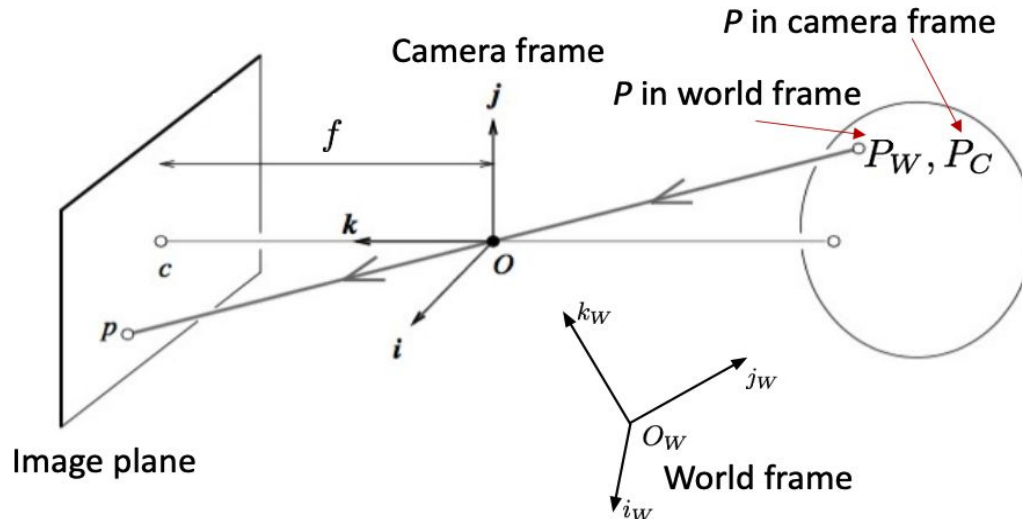
- Goal: project world points onto the camera image
- Assumption: we have a pinhole camera model (all results also hold under thin lens model, assuming camera is focused at ∞)
 - “frame”: coordinate system
 - “Camera frame”: origin at the lens center

And the world frame
has an origin
somewhere else



Perspective Projection

- Steps
 - 1. Map P_c into p (image plane)
 - 2. Map p into (u, v) pixel coordinates
 - 3. Transform P_w into P_c



P_w and P_c are the same point, but in different frame (coordinate system) with different origins

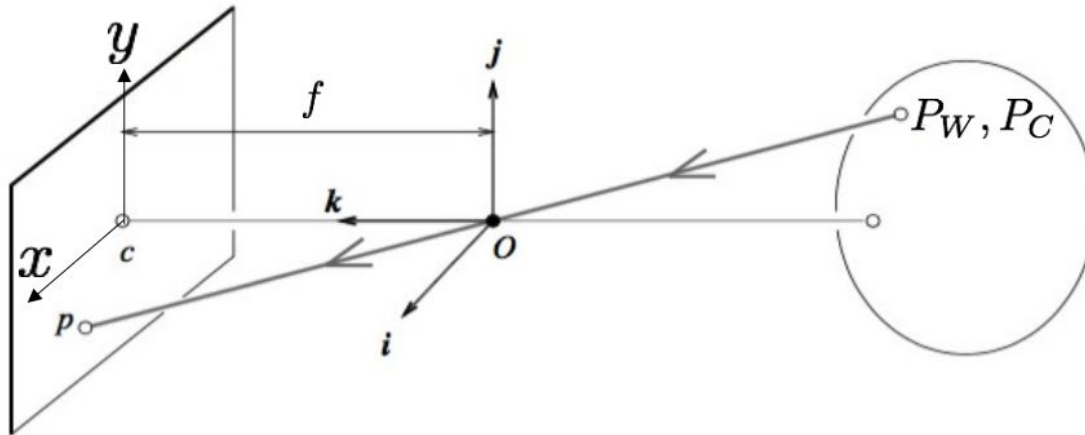
Perspective Projection

- Steps

- 1. Map $P_c = (X_c, Y_c, Z_c)$ into $p = (x, y)$ (image plane)
- We already have

$$\begin{cases} x = f \frac{X_c}{Z_c} \\ y = f \frac{Y_c}{Z_c} \end{cases}$$

why?

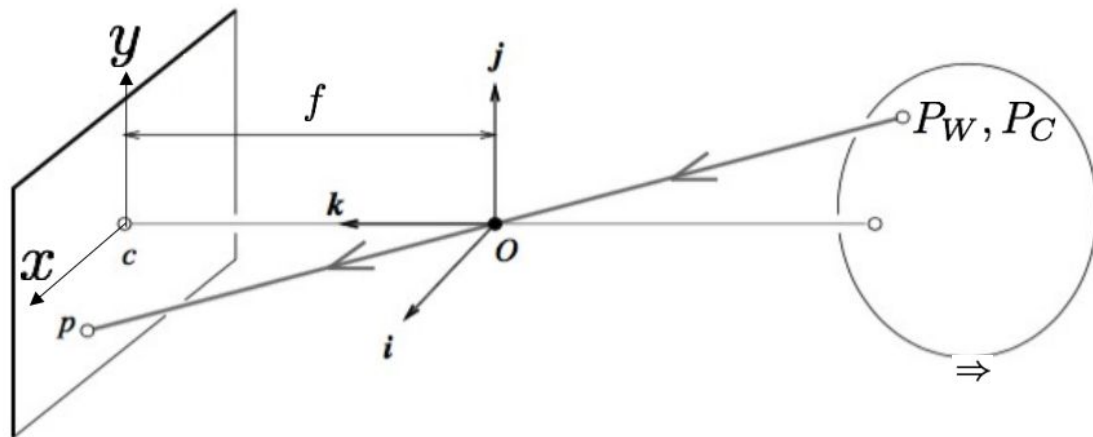


Perspective Projection

- Steps

- 1. Map $P_c = (X_C, Y_C, Z_C)$ into $p = (x, y)$ (image plane)
- We already have

$$\begin{cases} x = f \frac{X_C}{Z_C} \\ y = f \frac{Y_C}{Z_C} \end{cases}$$



Remember P, O, p are collinear (on the same line)

$$\overline{Op} = \lambda \overline{OP}$$

$$\begin{cases} x = \lambda X_C \\ y = \lambda Y_C \\ z = \lambda Z_C \end{cases}$$

$$z = f$$

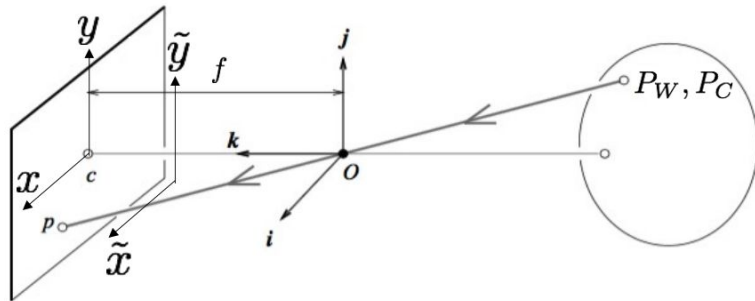
$$\lambda = \frac{x}{X_C} = \frac{y}{Y_C} = \frac{z}{Z_C}$$

$$\begin{cases} x = f \frac{X_C}{Z_C} \\ y = f \frac{Y_C}{Z_C} \end{cases}$$

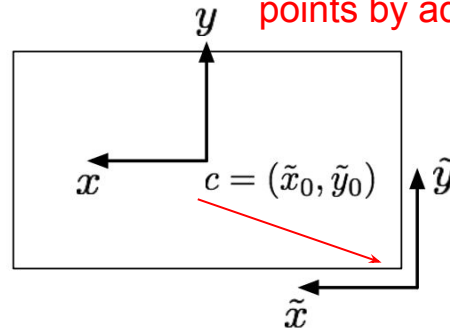
Perspective Projection

- Steps
 - 2. Map $p = (x, y)$ into (u, v) pixel coordinates
 - **Convert the coordinates to the corners of the image plane**

$$\begin{cases} x = f \frac{X_C}{Z_C} \\ y = f \frac{Y_C}{Z_C} \end{cases} \quad \tilde{x} = f \frac{X_C}{Z_C} + \tilde{x}_0, \quad \tilde{y} = f \frac{Y_C}{Z_C} + \tilde{y}_0,$$



Suppose c is the known center of the image plane, we can simply translate the points by adding c



Perspective Projection

- Steps
 - 2. Map $p = (x, y)$ into (u, v) pixel coordinates
 - Convert the image coordinates to **pixel coordinates**
 - Simply change the scale of the coordinates

We already have $\tilde{x} = f \frac{X_C}{Z_C} + \tilde{x}_0, \quad \tilde{y} = f \frac{Y_C}{Z_C} + \tilde{y}_0,$

Number of pixels per unit distance in image coordinates

$$u = k_x \tilde{x} = \underbrace{k_x f}_{\alpha} \frac{X_C}{Z_C} + \underbrace{k_x \tilde{x}_0}_{u_0}$$
$$v = k_y \tilde{y} = \underbrace{k_y f}_{\beta} \frac{Y_C}{Z_C} + \underbrace{k_y \tilde{y}_0}_{v_0}$$

$\Rightarrow$

$$\begin{aligned} u &= \alpha \frac{X_C}{Z_C} + u_0 \\ v &= \beta \frac{Y_C}{Z_C} + v_0 \end{aligned}$$

Nonlinear transformation

Homogeneous coordinates

- We want to represent the transformation as a linear mapping, so we need homogeneous coordinates

From euclidean coordinates to homogeneous coordinates. Basically you have one more dimension

$$\begin{pmatrix} x \\ y \end{pmatrix} \Rightarrow \lambda \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} \quad \begin{pmatrix} x \\ y \\ z \end{pmatrix} \Rightarrow \lambda \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

Perspective Projection

- Projection can be equivalently written in homogeneous coordinates

Focal length + Principal point offset

$$\begin{bmatrix} \alpha & 0 & u_0 \\ 0 & \beta & v_0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \begin{pmatrix} X_c \\ Y_c \\ Z_c \\ 1 \end{pmatrix} = \begin{pmatrix} \alpha X_c + u_0 Z_c \\ \beta Y_c + v_0 Z_c \\ Z_c \end{pmatrix}$$

Camera matrix/
Matrix of intrinsic parameters

P_c in homogeneous coordinates

Homogeneous pixel coordinates

$$p^h = [K \quad 0_{3 \times 1}] P_C^h$$

Perspective Projection

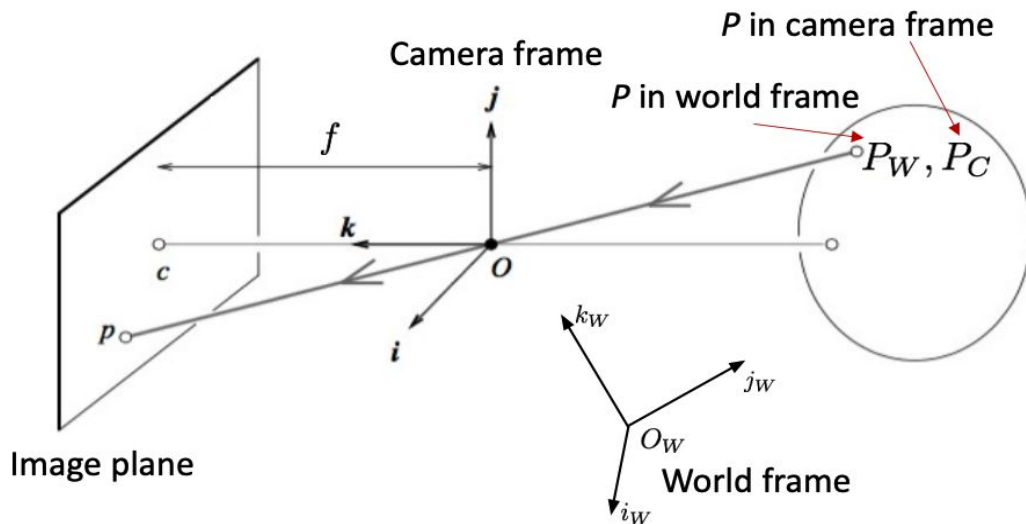
- Projection can be equivalently written in homogeneous coordinates

Point p in homogeneous pixel coordinates $\nearrow p^h = [K \quad 0_{3 \times 1}] P_C^h \nwarrow$ Point P_c in homogeneous camera coordinates

Perspective Projection

- Steps

- 1. Map P_c into p (image plane) ✓
- 2. Map p into (u, v) pixel coordinates ✓
- 3. Transform P_w into P_c

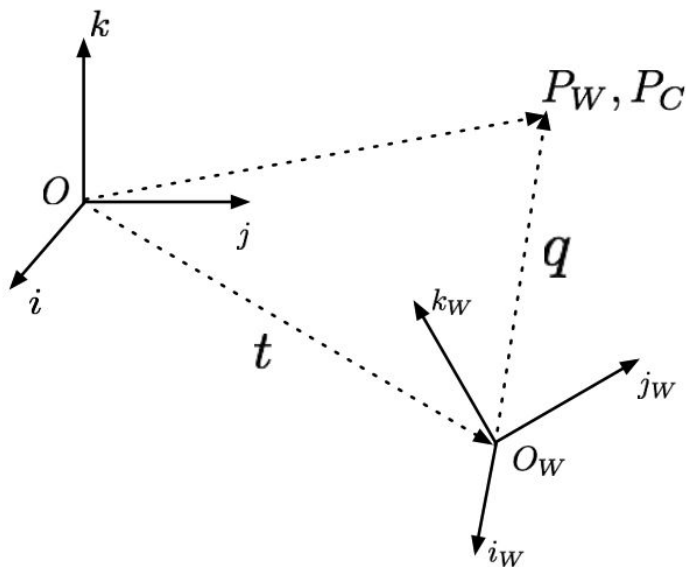


$$p^h = [K \quad 0_{3 \times 1}] P_C^h$$

Perspective Projection

- Steps

- 3. Transform P_W into P_C



$$P_C = t + q$$

$$q = R P_W$$

Rotation + Translation:
Rigid transformation

where R is the rotation matrix
relating camera and world
frames

$$R = \begin{bmatrix} i_W \cdot i & j_W \cdot i & k_W \cdot i \\ i_W \cdot j & j_W \cdot j & k_W \cdot j \\ i_W \cdot k & j_W \cdot k & k_W \cdot k \end{bmatrix}$$

$$\Rightarrow P_C = t + R P_W$$

Perspective Projection

- Steps

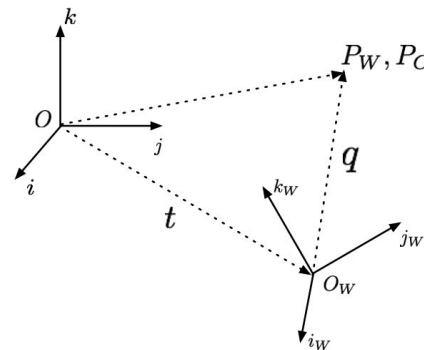
- 3. Transform P_W into P_C
 - In **homogeneous coordinates**

$$\Rightarrow P_C = t + R P_W$$

$$\begin{pmatrix} P_C \\ 1 \end{pmatrix} = \begin{bmatrix} R & t \\ 0_{1 \times 3} & 1 \end{bmatrix} \begin{pmatrix} P_W \\ 1 \end{pmatrix}$$

Point P_C in homogeneous coordinates

Point P_W in homogeneous coordinates



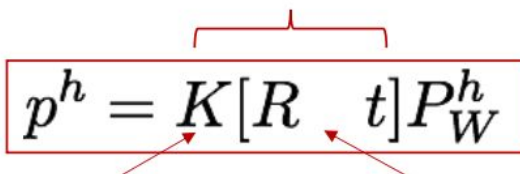
Perspective Projection

- Collecting all results

$$p^h = [K \quad 0_{3 \times 1}] P_C^h = K [I_{3 \times 3} \quad 0_{3 \times 1}] \begin{bmatrix} R & t \\ 0_{1 \times 3} & 1 \end{bmatrix} P_W^h$$

- Hence

Projection matrix M


$$p^h = K [R \quad t] P_W^h$$

Intrinsic parameters Extrinsic parameters

R: rotation
t: translation
Rt: extrinsic matrix

Degree of freedom:
K: 4
Rt: 6

Perspective Projection

- Collecting all results

$$p^h = [K \quad 0_{3 \times 1}] P_C^h = K [I_{3 \times 3} \quad 0_{3 \times 1}] \begin{bmatrix} R & t \\ 0_{1 \times 3} & 1 \end{bmatrix} P_W^h$$

- Hence

The diagram shows the equation $p^h = K[R \quad t]P_W^h$ enclosed in a red box. Above the box, a bracket labeled "Projection matrix M " spans the $[R \quad t]$ term. Below the box, two red arrows point to K and $[R \quad t]$, labeled "Intrinsic parameters" and "Extrinsic parameters" respectively. To the left of the equation is the vector $s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}$ and to the right is the vector $\begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$.

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \quad \begin{matrix} \text{Projection matrix } M \\ \hline p^h = K[R \quad t]P_W^h \\ \hline \text{Intrinsic parameters} \quad \text{Extrinsic parameters} \end{matrix} \quad \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

R: rotation
t: translation
Rt: extrinsic matrix

Degree of freedom:
K: 4
Rt: 6

Perspective Projection

- In simulators like Carla, the camera parameters are provided (or set by users)

Blueprint attribute	Type	Default	Description
bloom_intensity	float	0.675	Intensity for the bloom post-process effect, 0.0 for disabled.
fov	float	90.0	Horizontal field of view in degrees.
fstop	float	1.4	Opening of the camera lens. Aperture is 1/fstop with typical lens going down to f/1.2 (larger opening). Larger fstop values result in a shallower depth of field.
image_size_x	int	800	Image width in pixels.
image_size_y	int	600	Image height in pixels.
iso	float	100.0	The camera sensor sensitivity.
gamma	float	2.2	Target gamma value of the camera.
lens_flare_intensity	float	0.1	Intensity for the lens flare post-process effect, 0.0 for disabled.
sensor_tick	float	0.0	Simulation seconds between sensor captures (ticks).
shutter_speed	float	200.0	The camera shutter speed in seconds (1.0/s).

Projection matrix M

$$p^h = K[R \quad t]P_W^h$$

Intrinsic parameters Extrinsic parameters

R: rotation
t: translation
Rt: extrinsic matrix

Perspective Projection

- In simulators like Carla, the camera parameters are provided (or set by users)
 - No focal length? Only horizontal field of view?

Blueprint attribute	Type	Default	Description
bloom_intensity	float	0.675	Intensity for the bloom post-process effect, 0.0 for disabled.
fov	float	90.0	Horizontal field of view in degrees.
fstop	float	1.4	Opening of the camera lens. Aperture is 1/fstop with typical lens going down to f/1.2 (larger opening). Larger fstop values result in a shallower depth of field.
image_size_x	int	800	Image width in pixels.
image_size_y	int	600	Image height in pixels.
iso	float	100.0	The camera sensor sensitivity.
gamma	float	2.2	Target gamma value of the camera.
lens_flare_intensity	float	0.1	Intensity for the lens flare post-process effect, 0.0 for disabled.
sensor_tick	float	0.0	Simulation seconds between sensor captures (ticks).
shutter_speed	float	200.0	The camera shutter speed in seconds (1.0/s).

Projection matrix M

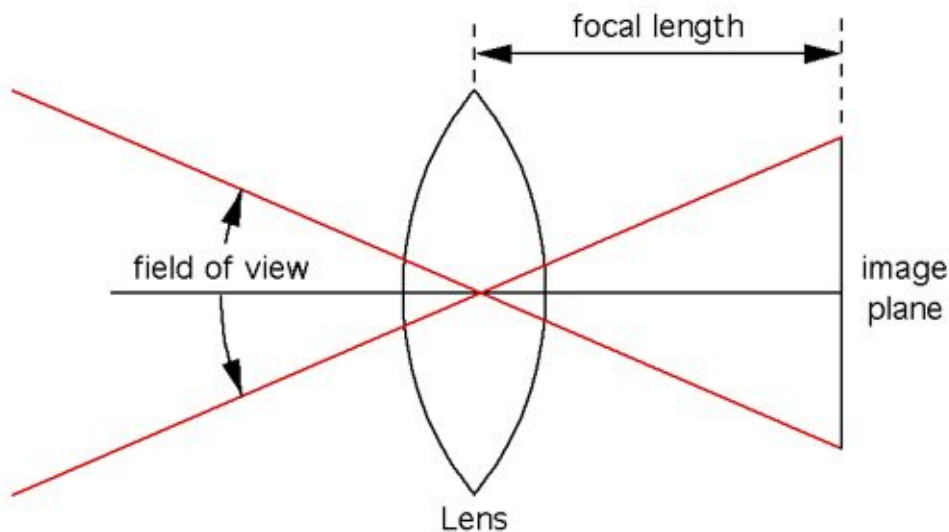
$$p^h = K[R \quad t]P_W^h$$

Intrinsic parameters Extrinsic parameters

R: rotation
t: translation
Rt: extrinsic matrix

Perspective Projection

- In simulators like Carla, the camera parameters are provided (or set by users)
 - No focal length? Only horizontal field of view?
 - We can compute that



horizontal field of view = $2 \operatorname{atan}(0.5 \text{ width} / \text{focallength})$

Python code:

```
Focus_length = ImageSizeX / (2 * tan(CameraFOV * π / 360))  
Center_X = ImageSizeX / 2  
Center_Y = ImageSizeY / 2
```

Perspective Projection

- In simulators like Carla, the camera parameters are provided (or set by users)
 - No focal length? Only horizontal field of view?
 - Here is the full intrinsic in Carla

Python code:

```
Focus_length = ImageSizeX / (2 * tan(CameraFOV * π / 360))  
Center_X = ImageSizeX / 2  
Center_Y = ImageSizeY / 2
```

```
K = [[f, 0, Cu],  
     [0, f, Cv],  
     [0, 0, 1 ]]
```

Perspective Projection

- In simulators like Carla, the camera parameters are provided (or set by users)
 - No focal length? Only horizontal field of view?
 - Here is the full intrinsic in Carla

Python code:

```
Focus_length = ImageSizeX / (2 * tan(CameraFOV * π / 360))  
Center_X = ImageSizeX / 2  
Center_Y = ImageSizeY / 2
```

```
K = [[f, 0, Cu],  
     [0, f, Cv],  
     [0, 0, 1 ]]
```

How to do perspective projection? Given $p2d = [u, v, 1]$, how to get world point $P = [X, Y, Z]$?

Perspective Projection

- In simulators like Carla, the camera parameters are provided (or set by users)
 - No focal length? Only horizontal field of view?
 - Here is the full intrinsic in Carla

Python code:

```
K = [[f, 0, Cu],  
      [0, f, Cv],  
      [0, 0, 1 ]]
```

```
def compute_intrinsic(img_width, img_height, fov):  
    """Compute intrinsic matrix."""  
    intrinsic = np.identity(3)  
    intrinsic[0, 2] = img_width / 2.0  
    intrinsic[1, 2] = img_height / 2.0  
    intrinsic[0, 0] = intrinsic[1, 1] = img_width / (2.0 * np.tan(fov *  
                                                                np.pi / 360.0))  
  
    return intrinsic
```

How to do perspective projection? Given $p2d = [u, v, 1]$, how to get world point $P = [X, Y, Z]$?
 R_t (camera extrinsic) is known in 3D simulation at all times

$$P = (\text{inv}(K) * p2d) * \text{depth} * R_t$$

Agenda - Camera Models & Camera Calibration

- Objectives
 - Learn about cameras, camera models and camera calibration
 - So you can put what you see/what the camera sees into the 3D world!
- The tasks
 - Camera models
 - Perspective projection
 - **Camera calibration**
 - A note on 3D reconstruction

Camera Calibration

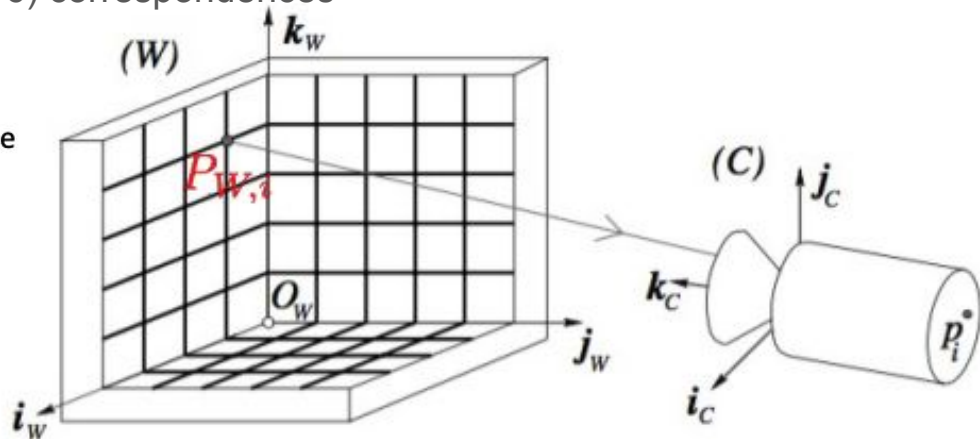
- The target for camera calibration is to get the **intrinsic and extrinsic parameters** of the camera

- We need to know 10* parameters (4 for K, 3 for R, 3 for t)
- Strategy
 - We need enough (at least 6) correspondences

$$p_i \leftrightarrow P_{W,i}$$

$P_{W,1}, P_{W,2}, \dots, P_{W,n}$ with **known** positions in world frame

$p_1, p_2, \dots, p_n$ with **known** positions in image frame



* or 11 if we include skewness

Camera Calibration

- The target for camera calibration is to get the **intrinsic and extrinsic parameters** of the camera

- We need to know 10* parameters (4 for K, 3 for R, 3 for t)

- Strategy

- We need enough (at least 6) correspondences

$$p_i \leftrightarrow P_{w,i}$$

- Method

- With the help of a chessboard and OpenCV...

- It is easy to compute correspondences for some patterns

- https://docs.opencv.org/4.x/dc/dbb/tutorial_py_calibration.html



`cv.findChessboardCorners()`.

`cv.calibrateCamera()`

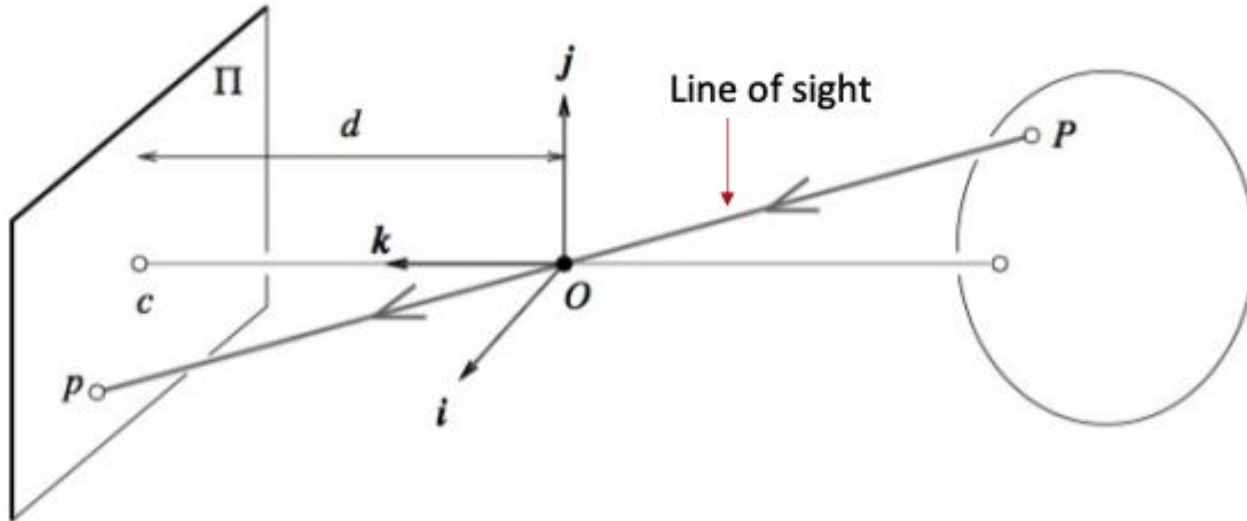
* or 11 if we include skewness

Agenda - Camera Models & Camera Calibration

- Objectives
 - Learn about cameras, camera models and camera calibration
 - So you can put what you see/what the camera sees into the 3D world!
- The tasks
 - Camera models
 - Perspective projection
 - Camera calibration
 - **A note on 3D reconstruction**

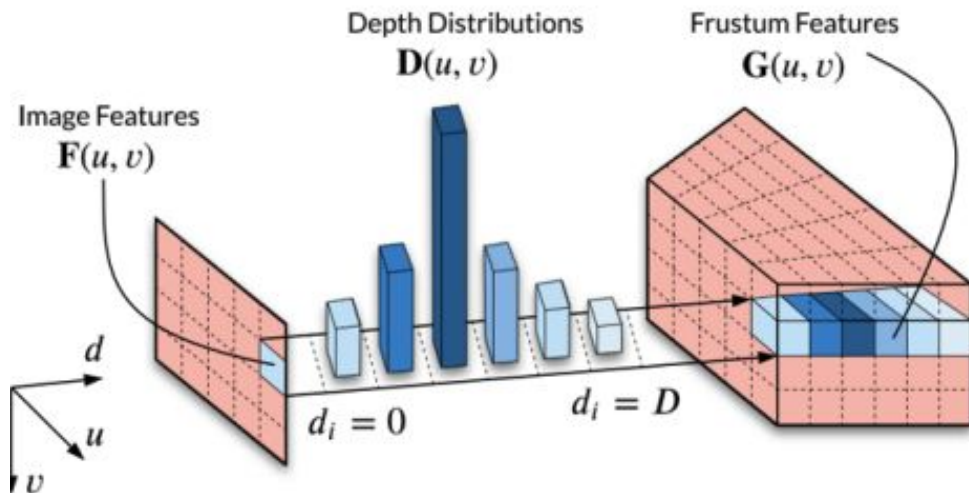
Measuring depth - 3D reconstruction

- If the camera is calibrated (intrinsic and extrinsic known), given image point p , can we get world point P in 3D world?
 - No! We can only know P is on the line OP



Measuring depth - 3D reconstruction

- If the camera is calibrated (intrinsic and extrinsic known), given image point p , can we get world point P in 3D world?
 - No! We can only know P is on the line OP
 - Remember the 2D feature to 3D frustum project in BEV perception



Measuring depth - 3D reconstruction

- If the camera is calibrated (intrinsic and extrinsic known), given image point p , can we get world point P in 3D world?
 - No! We can only know P is on the line OP
 - So how?
 - Stereo vision
 - Two cameras, relative pose is known (like our eyes)
 - Structure from motion
 - Same camera, multiple images
 - LiDAR!

Measuring depth - 3D reconstruction

- If the camera is calibrated (intrinsic and extrinsic known), given image point p , can we get world point P in 3D world?
 - No! We can only know P is on the line OP
 - But we can still get from **image plane to the ground plane** (BEV!!)
 - Need to know the ground plane's parameters
 - Homography matrix = $K_2 * R_{21} * \text{inv}(K_1)$
 - This allow you to..

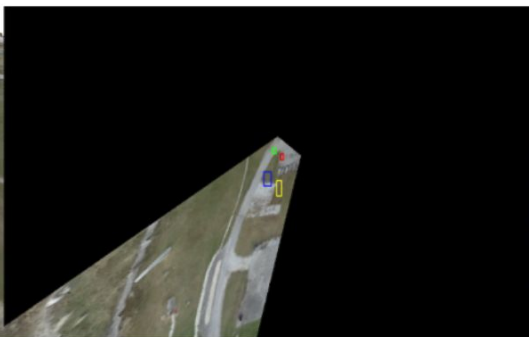
Intrinsics of the image camera and
the ground plane camera

Measuring depth - 3D reconstruction

- If the camera is calibrated (intrinsic and extrinsic known), given image point p , can we get world point P in 3D world?
 - No! We can only know P is on the line OP
 - But we can still get from **image plane to the ground plane** (BEV!!)
 - Need to know the ground plane's parameters
 - Homography matrix = $K_2 * R_{21} * \text{inv}(K_1)$
 - This allow you to..



Original camera

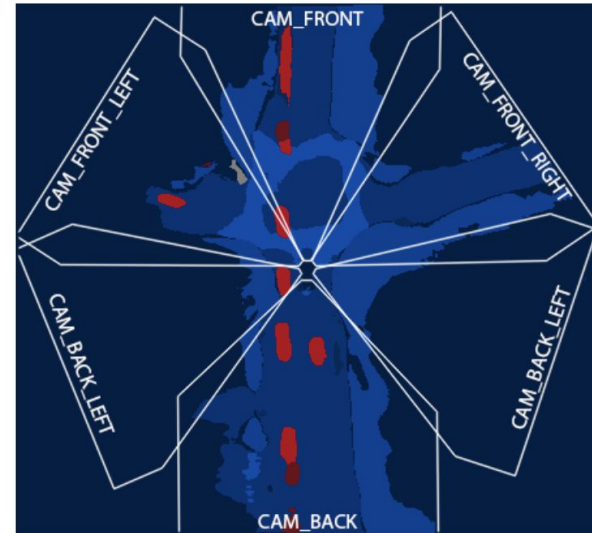


Warped using computed homography
between the camera and top-down view



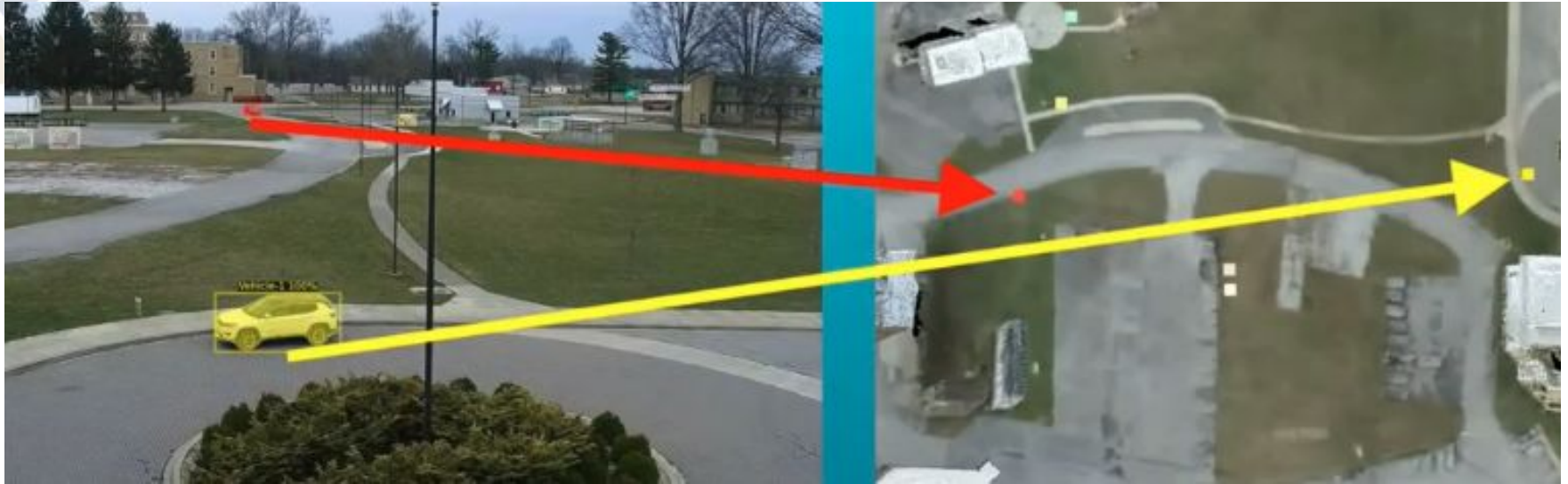
Top-down view camera

Recap: Bird-Eye-View Perception



Drivable	Dark Blue
Ped. crossing	Blue
Walkway	Light Blue
Carpark	Dark Blue
Car	Red
Truck	Dark Red
Bus	Purple
Trailer	Dark Brown
Constr. veh.	Brown
Pedestrian	Yellow
Motorcycle	Teal
Bicycle	Green
Traffic cone	Orange
Barrier	Grey

- Put object tracking results to a top-down map



Camera 1

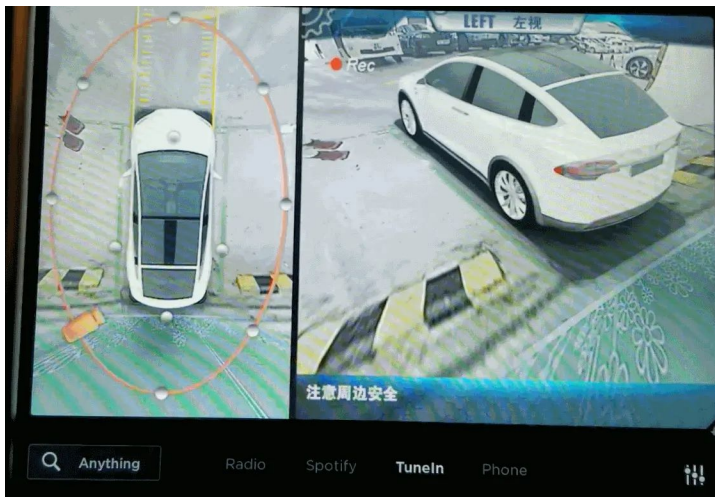
Camera 2



We recreate the static scene and dynamic trajectories from real-world videos

Bird-Eye-View Perception

- Existing BEV applications



Agenda - Camera Models & Camera Calibration

- Objectives

- Learn about cameras, camera models and camera calibration
- So you can put what you see/what the camera sees into the 3D world!

- The tasks

- Camera models
- Perspective projection
- Camera calibration
- A note on 3D reconstruction

Reading Material

- Camera calibration tutorial
 - https://docs.opencv.org/4.x/dc/dbb/tutorial_py_calibration.html
 - https://boofcv.org/index.php?title=Tutorial_Camera_Calibration
 - https://github.com/zhengthomastang/Cal_PnP
- EPro-PnP: Generalized End-to-End Probabilistic Perspective-n-Points for Monocular Object Pose Estimation, CVPR 2022 best paper
 - Calibrate objects in the image



Next: Visual recognition, object detection and Tracking!



Thank you!